Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000. Digital Object Identifier 10.1109/ACCESS.2026.0000000

Blockchain-Assisted Zero-Trust Architecture for Secure and Scalable Industrial Internet of Things (IIoT)

[Author Name(s)]
[Department, Institution, City, Country]

Corresponding author: [Name] (e-mail: [address]). This work was supported in part by [funding source and grant number].

ABSTRACT Industrial IoT (IIoT) networks need continuous, per-request verification, but placing a blockchain on the authentication path adds seconds of latency and saturates ledger capacity. We propose BC-ZTA, a three-tier blockchain-assisted zero-trust architecture in which edge policy enforcement points decide from cached, Merkle-verified ledger state, while identity, trust and audit updates are written asynchronously to a permissioned Hyperledger Besu (IBFT 2.0) network through four Solidity contracts. We implemented the policy enforcement point and three baselines and measured them end to end on a four-validator Besu network (three repetitions, shared two-vCPU host). At 50 devices BC-ZTA answers in 7.4 ms on average, compared with 48.2 ms for a centralised directory with an emulated WAN link and 10257 ms when every request is an on-chain transaction; the synchronous design saturates at about 47 requests per second, whereas BC-ZTA served 200 devices with no failures and about one on-chain write per second. Event-driven cache invalidation enforces a revocation in 1.2 s on average, and all 1050 forged, replayed, stale, unregistered, tampered or policy-violating requests were rejected. Merkle batch anchoring cuts audit gas per record by about 1351×. Limitations are the shared single host, the emulated WAN delay and the emulated device population.

INDEX TERMS Access control, blockchain, Hyperledger Besu, industrial Internet of Things, scalability, smart contracts, zero trust.

I. INTRODUCTION

A. IIoT Security Background

The Industrial Internet of Things (IIoT) interconnects programmable logic controllers, sensors, actuators, edge gateways and cloud analytics in smart manufacturing, energy and process-control plants. Historically these operational-technology (OT) networks relied on a hardened perimeter and implicit trust between devices on the same segment. As OT converges with IT and remote maintenance becomes routine, that perimeter dissolves, and a single compromised vendor laptop or sensor can become a foothold for lateral movement [1], [2]. Zero Trust Architecture (ZTA), formalised in NIST SP 800-207 [3], replaces location-based trust with per-request verification, and recent systematic reviews confirm its rapid uptake across domains [4]–[6].

B. Security Challenges

Four properties make IIoT hard to secure. (i) *Constrained devices* cannot run heavy cryptography or continuous agents; (ii) *heterogeneity* of protocols and vendors complicates uniform identity management [7], [8]; (iii) *hard real-time budgets* leave only milliseconds for a security decision; and (iv) *scale*, with thousands of devices that join, rotate keys and are revoked continuously. A centralised identity or policy server is both a single point of failure and a high-value target, and conventional logs held by that same operator are not independently trustworthy [5].

C. Zero-Trust Requirements

Following NIST SP 800-207 [3] and critical analyses of ZTA [9], an IIoT deployment needs: **R1** strong, per-device cryptographic identity; **R2** per-request authorisation by a policy decision point (PDP) enforced by a policy enforcement point (PEP); **R3** least-privilege, short-lived sessions; **R4** continuous trust assessment from behavioural and posture signals [10]; **R5** micro-segmentation and containment; and **R6** tamper-evident audit. R1–R5 must be satisfied within control-loop latency budgets.

D. Role of Blockchain

A permissioned blockchain offers a shared, tamper-evident source of truth among mutually distrusting plant, vendor and integrator domains: a decentralised identity registry, policy and reputation state governed by smart contracts, and an immutable audit trail. Prior work has applied it to zero-trust IoT access [11], [12], cross-domain digital-twin access [13], IIoT data protection [14] and contract-based access control [15], [16].

E. Research Gap

Despite this progress, three gaps remain. First, many designs place the ledger *on the request path*; a published Ethereum-based zero-trust framework, for instance, reports latency growing from 49.33 ms to 74.0 ms and throughput falling from 50 to 30.77 req/s [17]. Second, scalability is seldom evaluated for authentication and authorisation workloads, with notable exceptions that rely on sharding [18]; most studies evaluate accuracy or encryption cost [19], [20]. Third, continuous tamper-proof audit logging, repeatedly named as an open gap [5], is expensive when every event is written on-chain.

F. Research Objectives

We aim to (O1) design a blockchain-assisted ZTA whose per-request decisions do not wait for consensus; (O2) bound the resulting staleness of revocation and trust state; (O3) make audit anchoring cost sub-linear in the event rate; and (O4) evaluate latency, on-chain load, gas cost and revocation delay against three baselines.

G. Contributions

- **Architecture.** A three-tier BC-ZTA in which the PEP/PDP decide from edge-cached, Merkle-verified ledger state, so the blockchain is *off* the request path (Section III).
- **Smart contracts.** Four Solidity contracts (identity registry with epoch commitments, quorum-based trust reputation, ABAC access control, audit log with Merkle batch anchoring), compiled and unit-tested with 15 passing tests.

- **End-to-end prototype and measurements.** A running PEP/PDP service, three baselines (centralised PKI-IAM, local ABAC, blockchain-synchronous) and a load generator, evaluated on a four-validator Hyperledger Besu IBFT 2.0 network in three repetitions: latency and goodput up to 200 devices, revocation delay, security tests and gas (Sections 5–6).
- **Bounded-staleness analysis.** Event-driven invalidation reduces mean revocation delay from 16.5 s to 1.5 s (Section VI-E).

The remainder of the paper reviews related work (Section II), presents the architecture (Section III), the use case and threat model (Section IV), the methodology (Section V), results (Section VI) and conclusions (Section VII).

II. RELATED WORK

A. Blockchain-Based IIoT Security

Essaid and Ju combine blockchain, edge computing and smart contracts for IIoT data integrity and threat detection [14]. Hyper-IIoT proposes a smart-contract-inspired access-control scheme for resource-constrained devices [15], and formal model checking of IoT access-control contracts has been explored [16]. Lightweight authentication using blockchain and Chebyshev chaotic maps targets cross-domain IIoT [21]. For identity, Asim *et al.* use epoch-based batch registration with sparse Merkle trees to support up to 7,000 devices over 20 fog nodes with under 5 MB on-chain overhead [8], and IOTA-based decentralised identifiers have been prototyped on Raspberry Pi 4 devices, with CPU load peaking during DID generation [22]. DAG ledgers have been proposed for scalability and privacy in intelligent buildings [23], and a smart-city trust framework reports 450–500 tps with a lightweight adaptive PoS [24].

B. Zero-Trust IIoT Security

Federici *et al.* design a zero-trust remote-access architecture for IIoT [1], and Zanasi *et al.* a flexible architecture combining micro-segmentation, SDN and a central policy layer [2]. Ma and Chiu propose a risk-based access-control engine with continuous neighbour-trust assessment for IoT [10]; Mthethwa *et al.* address non-human identities [7]. Broader treatments include a ZTA framework for next-generation IoT [25], trust-driven adaptive edge resource management that lowers latency by 28.7% against cloud processing [26], and zero-trust foundation models for IoT AI [27]. Surveys summarise applications and challenges [28].

C. Blockchain-Assisted Zero Trust

Aleisa couples blockchain-enabled ZTA with post-quantum and zero-knowledge techniques, reporting 98% privacy preservation [11]. Nie *et al.* use smart contracts and inner-product encryption for 6G IoT access control, with 14% higher encryption efficiency and 18% lower access latency [12]. Sharma *et al.* add federated transfer learning for Industry 5.0 [19], and Mishra *et al.* a zero-trust digital twin with hash-chained lightweight blockchain auditing and deep-learning intrusion detection [20]. Salim *et al.* reach 2,500 TPS at 700 nodes through DRL-driven sharding [18]. Domain variants exist for FinTech [17], clinical trials with T-PBFT [29] and FPGA supply chains [30].

D. Comparative Analysis

Table 1 compares representative works on four criteria taken from their reported content.

TABLE 1. COMPARISON OF RELATED WORK

Work	Blockchain	Zero trust	IIoT focus	Scale eval.
[18]	Yes	Yes	No	Yes
[11]	Yes	Yes	No	No
[12]	Yes	Yes	No	Part
[19]	Yes	Yes	Part	No
[20]	Yes	Yes	Yes	Part
[13]	Yes	Part	Yes	Part
[15]	Yes	No	Yes	No
[8]	Yes	No	Yes	Yes
[10]	No	Yes	No	No
[2]	No	Yes	Yes	No
This work	Yes	Yes	Yes	Yes

Yes = addressed; Part = partly; No = not addressed. Authors' reading of each paper's reported content; "Scale eval." means device/node scaling reported.

Three observations follow. Most blockchain-based zero-trust designs target generic IoT rather than IIoT; only two of the listed works report device or node scaling [8], [18]; and none of the listed works both reports scaling and quantifies the staleness introduced by off-path state. The last point is where this paper concentrates its evaluation.

E. Research Gap

No surveyed work simultaneously (a) keeps the ledger off the per-request path, (b) quantifies the revocation-staleness this causes and how to bound it, and (c) amortises audit cost via Merkle anchoring for an IIoT workload. This paper targets that combination.

III. PROPOSED ARCHITECTURE

A. Design Objectives

D1 decisions from local state in milliseconds; **D2** no single trusted party for identity, policy or logs; **D3** bounded staleness of cached state; **D4** write load that grows slower than the request rate; **D5** compatibility with constrained devices (ECDSA signatures, hash proofs only).

B. Architecture Overview

Fig. 1 shows three tiers. Devices form Layer I (IIoT and edge). Layer II is the zero-trust security layer at the edge gateway, hosting the PEP, PDP and trust engine. Layer III is the permissioned blockchain, whose identity, trust, access-control and audit services are implemented as smart contracts. Gateways cache Layer III state, so Layer II never waits for consensus.

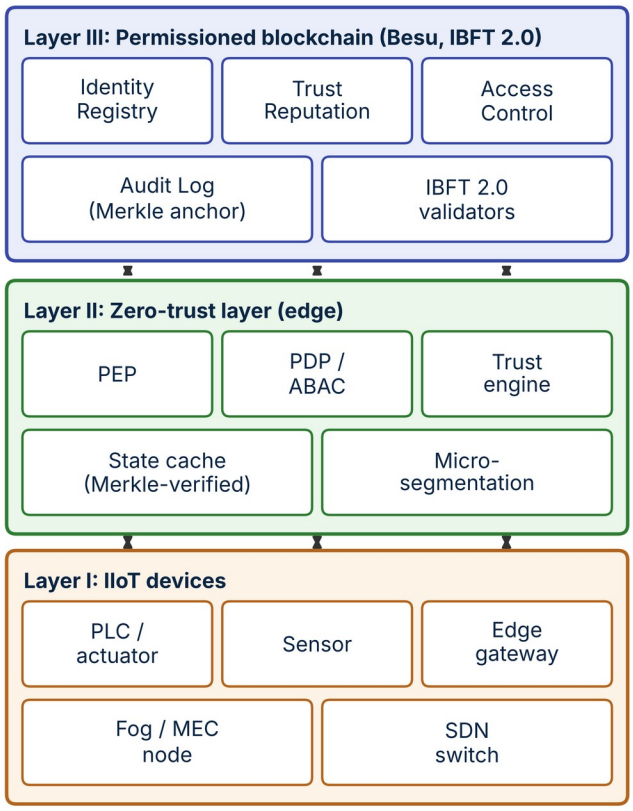


Fig. 1. Proposed BC-ZTA architecture: devices, edge/ZT layer, blockchain layer and contract services.

C. IIoT/Edge Layer

Sensors, actuators and PLCs hold an ECDSA key pair bound to a DID and sign each request with a timestamp and nonce. Edge gateways terminate sessions, host the PEP, keep a local cache of the identity, trust and policy state, and forward anomaly telemetry to the trust engine.

D. Zero-Trust Security Layer

The PEP intercepts every request. The PDP verifies the signature, checks the DID against a cached Merkle-verified registry entry, retrieves the trust score and policy, and returns allow/deny. Micro-segmentation is enforced by permitting only flows that hold a valid, unexpired session [2]. A trust engine converts behavioural features (rate, protocol conformance, firmware attestation) into score proposals.

E. Blockchain Layer

A permissioned Hyperledger Besu network with IBFT 2.0 consensus is operated by plant, integrator and vendor validators; with n validators it tolerates $f = \lfloor (n-1)/3 \rfloor$ Byzantine nodes. Gateways are read-only clients that subscribe to events; only authorised registrars, validators and the policy administrator send transactions.

F. Identity Management

Each device is a DID with a public-key hash and device class. Registrations are submitted by a plant-authorised registrar and may be grouped into *epochs*: the registrar commits a Merkle root of an epoch batch and devices later prove membership with a logarithmic-size proof, an approach inspired by epoch-based IIoT identity schemes [8]. Key rotation and revocation are single transactions that emit events.

G. Trust Management

Scores lie in [0,100] with an initial value of 70. A score changes only when a quorum of validators proposes the same value, and updates are rate-limited. An anomaly flag immediately subtracts 20 points (floored at 0) pending re-evaluation, giving fast conservative reaction.

H. Access Control

Policies are attribute-based: for each resource the administrator defines allowed device-class range, minimum trust score and session TTL. A request is granted iff the DID is active, the class is in range and the trust score meets the threshold; the grant yields a session identifier that expires after the TTL or on explicit revocation.

I. Smart Contracts

IdentityRegistry (register, update key, revoke, epoch commit, membership check); *TrustReputation* (quorum voting, anomaly flag); *AccessControl* (policy, evaluate, session grant/revoke); *AuditLog* (per-event log, Merkle batch anchor, inclusion verification). Table 2 lists the interfaces, and source and tests are in the accompanying package.

TABLE 2. SMART-CONTRACT INTERFACES

Contract	Main functions	Writer
IdentityRegistry	registerDID, updateKey, revokeDID, commitEpoch; verifyMembership, isActive (views)	Plant authority
TrustReputation	updateScore (quorum), flagAnomaly; getScore (view)	Validators
AccessControl	setPolicy, revokeSession; evaluateRequest (view), grantSession, isSessionValid	Admin; any caller
AuditLog	logEvent, anchorBatch; verifyInBatch, getAnchor (views)	Open in prototype

"Open in prototype" means the function has no caller restriction; a deployment would restrict it to gateway accounts.

J. Continuous Verification

Fig. 2 summarises the workflow. Verification does not end at login: every request is re-evaluated against current cached trust, sessions are short-lived, and the gateway refreshes cached entries every *TTL* seconds or immediately on a *DIDRevoked* or *ScoreUpdated* event. Audit records are hashed into a batch whose Merkle root is anchored on-chain asynchronously (step 7), so logging never delays the response.

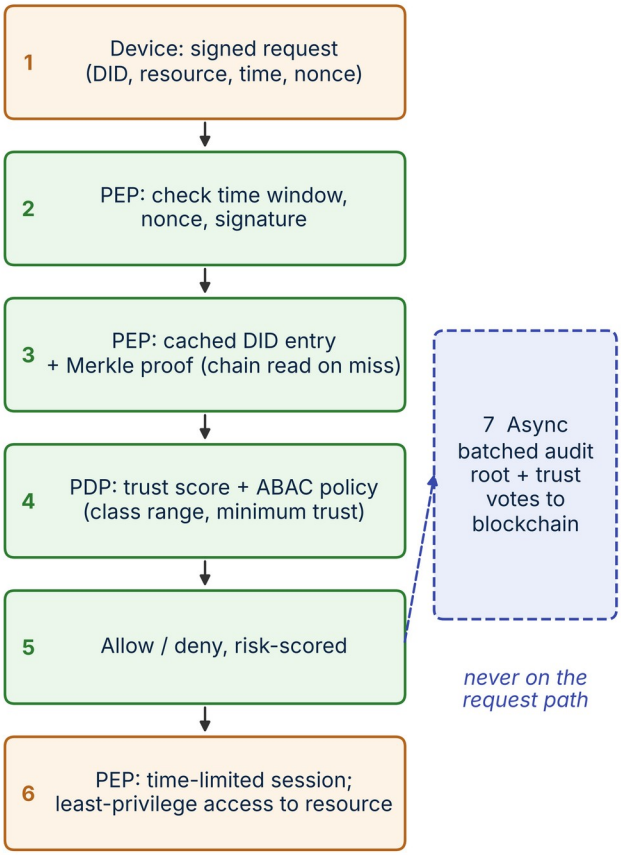


Fig. 2. Per-request verification workflow with cached state reads and asynchronous audit anchoring (step 7).

A request is r = (DID, resource, operation, t, n, σ), where t is a timestamp, n a nonce and σ the device's ECDSA signature over the other fields. The PEP accepts r only if (i) $|\text{now} - t| \le \Delta$ and n has not been seen within Δ, which bounds replay; (ii) the DID is active in the cached registry, proved by a Merkle path against the latest committed epoch root or by a direct chain read on a cache miss; (iii) σ verifies under the public key whose hash is registered; and (iv) the PDP finds the trust score and policy satisfied. Otherwise the request is denied and an anomaly event is queued. As a fail-closed rule, a gateway whose cached state is older than $2 \cdot TTL$ because it lost contact with the ledger refuses new sessions, while existing sessions run to their own expiry.

IV. USE CASE AND THREAT MODEL

A. Use Case

A smart factory has several production cells, each with PLCs and sensors behind one edge gateway; a remote vendor engineer needs time-limited maintenance access to one PLC class; a SCADA historian consumes sensor data. Cells, vendor and integrator are separate administrative domains that jointly operate validators.

B. Assets and Trust Assumptions

Assets are device identities, policy and trust state, session tokens and audit records. We assume a majority-honest validator set (at most f Byzantine), secure device key storage, and gateways that are trusted for enforcement but may be compromised individually.

C. Adversary Model

A Dolev–Yao network adversary can eavesdrop, replay and inject messages; may compromise a subset of devices (including their keys) and one gateway; and may include an insider with administrative credentials in one domain but not in a validator quorum.

D. Threats and Countermeasures

Table 3 maps threats to mechanisms.

TABLE 3. THREATS AND COUNTERMEASURES

Threat	Attack	Countermeasure
Spoofing	Forged device identity	DID + ECDSA signature; Merkle-verified registry entry
Replay	Re-sent request	Timestamp + nonce window at PEP; session TTL
Lateral movement	Compromised device probes peers	Per-resource ABAC, micro-segmentation, short sessions
Insider / tamper	Admin edits policy or logs	On-chain policy and Merkle-anchored audit; validator quorum
Trust poisoning	False score votes	Quorum votes, rate limit; non-validators rejected
Stale state	Use of revoked identity	Event-driven cache invalidation, bounded TTL
Ledger DoS	Flooding the chain	Off-path reads; batched writes; non-validators cannot write trust

E. Security Goals

Authenticity of devices, least-privilege authorisation, revocation within a bounded delay, non-repudiable tamper-evident audit, and availability of access decisions during ledger congestion.

V. IMPLEMENTATION AND EXPERIMENTAL METHODOLOGY

A. Besu Blockchain Network

We deployed a four-validator Hyperledger Besu v26.9.0 IBFT 2.0 network (f = 1, block period 2 s, chain ID 2026, zero base fee, London EVM) with Docker on a GitHub-hosted 2-vCPU Linux runner; all validators share that single host, so network delay is negligible and CPU is contended. A fresh network was started for each of 3 repetitions. Contracts were compiled with Solidity 0.8.20 (optimizer 200 runs, EVM "london"). Five scenarios were run with pre-signed transactions from 8 worker wallets: S1 sequential write confirmation, S2 read latency (500 *eth_call* evaluations of *AccessControl.evaluateRequest*), S3 per-event logging at 10–200 tx/s offered for 20 s, S4 batch anchoring at 1 tx/s, and S5 revocation visibility. Block observation was polled every 25 ms. The benchmark scripts, contracts and raw results are available from the authors on request.

B. Smart Contracts

The four contracts total about 370 lines. A Hardhat/Chai suite of 15 tests passes and covers registration, duplicate and unauthorised registration, revocation, key update, quorum scoring, score floor, rate limiting, non-validator rejection, access grant/deny, session expiry, epoch Merkle membership and audit batch inclusion proofs.

C. End-to-End Prototype and Load Generator

Besides the contract-level tests, we implemented the complete request path as running software (Node.js 20): a PEP/PDP service, a load generator and the baselines of Section V-D. Each device owns a secp256k1 key and signs *DID* | *resource* | *timestamp* | *nonce*; its key hash and device class are registered on-chain and its DID is part of one Merkle epoch commitment. The PEP rejects stale timestamps (>5 s) and repeated nonces, recovers the signer, checks it against the registered key hash, verifies the Merkle membership proof against the cached epoch root, and evaluates the ABAC policy (device class 0–1, trust score ≥ 50). In BC-ZTA, identity, trust and policy state are read from Besu with *eth_call* on a cache miss and cached for 30 s; a block-polling subscription (20 ms) invalidates an entry when a *DIDRevoked*, *KeyUpdated* or *ScoreUpdated* event is mined; every decision is hashed, and once per second the Merkle root of the pending batch is anchored with *AuditLog.anchorBatch*. The load generator emulates N = 10, 25, 50, 100, 200 devices, each sending one request per second with staggered phases, for 5 s of warm-up and a 20 s measurement window; requests are signed before the window so that the generator itself is not the bottleneck. The PEP, the baseline servers, the load generator and all four Besu validators run on the same 2-vCPU host.

D. Baselines

(B1) *Centralised PKI-IAM*: each request makes one lookup at a separate directory server, with an emulated WAN round trip of 40 ms plus 2 ms directory time (an in-process delay, not a real network). (B2) *ABAC without blockchain*: a local attribute store at the PEP. (B3) *Blockchain-only (sync)*: for every request the PEP reads the state with *eth_call* and sends an *AuditLog.logEvent* transaction (eight sender accounts) and replies only after the block containing it is observed. (P) *Proposed BC-ZTA* as in Section V-C. All four use the same request format, signature check and policy, so differences come from where state is read and written.

E. Experimental Parameters

Table 4 lists the experimental parameters; the PKI delay is an assumption, the other values are configuration of the prototype.

TABLE 4. EXPERIMENTAL PARAMETERS

Parameter	Value
Ledger	Besu v26.9.0, IBFT 2.0, 4 validators
Block period	2 s
Host	GitHub-hosted, 2 vCPU, 8 GB
Devices; request rate	10–200; 1 req/s each
Warm-up; window; runs	5 s; 20 s; 3
Cache TTL; anchor interval	30 s; 1 s
Emulated WAN RTT (B1)	40 ms
Freshness window; policy	5 s; class 0–1, trust ≥ 50
Revocation tests	8 devices × 3 runs per system; TTL 10 s

Only the PKI round trip is emulated; everything else is measured on running software.

F. Performance Metrics

(E1) End-to-end latency (mean, P95) from request send to decision, goodput (accepted requests per second), failures, and on-chain writes per second versus device count. (E2) Revocation enforcement delay: time from the revocation action until the PEP first denies the device. (E3) Security tests with 50 attempts each per run. (E4) Decision latency while the ledger is flooded with transactions. Also gas per operation and, from the micro-benchmark, confirmation latency and ledger throughput.

VI. RESULTS AND DISCUSSION

A. Authentication

Table 5 and Fig. 3 show end-to-end latency (mean of 3 runs). At 50 devices BC-ZTA answers in 7.4 ms on average (P95 22.3 ms), versus 48.2 ms for the centralised PKI-IAM, 4.4 ms for the ledger-free ABAC store and 10257 ms for the synchronous blockchain. BC-ZTA is therefore 85% faster than the centralised baseline, costs about 3.0 ms more than a local store (Merkle verification, hashing and audit bookkeeping), and is about 1381× faster than checking every request on-chain. Even at 10 devices, where the ledger is not saturated, the synchronous design needs 1426 ms because each request waits for a block.

TABLE 5. END-TO-END LATENCY (MS, MEAN OF 3 RUNS)

System	Mean 50	P95 50	Mean 200	P95 200
PKI-IAM	48.2	57.1	60.8	126.7
ABAC	4.4	8.8	12.0	60.5
BC-only	10257	15847	15047	27169
BC-ZTA	**7.4**	**22.3**	**54.7**	**208.6**

Columns give the number of devices; PKI-IAM includes an emulated 40 ms WAN round trip.

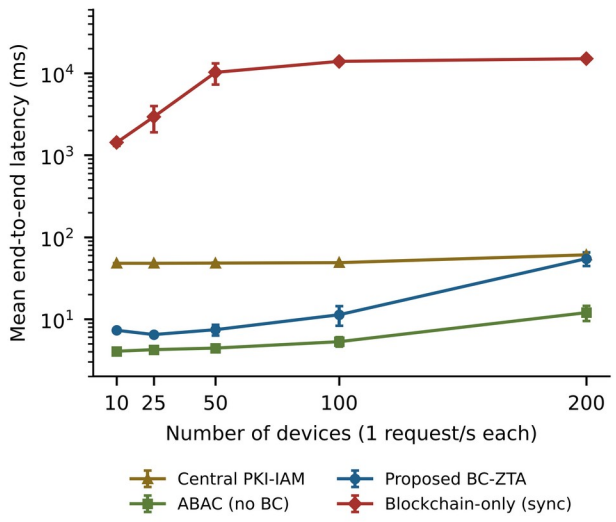


Fig. 3. Mean end-to-end latency versus number of devices (log scale; error bars: standard deviation of 3 runs).

B. Authorization

The proposed design makes policy decisions from the edge cache. The measured cache-hit ratio was 96% (one miss per device per 30 s TTL), so 50 devices caused only 50 chain reads for about 1,250 requests; each miss costs one batched *eth_call* of 5.4 ms in the micro-benchmark. The PEP used 0.26 CPU cores at 50 devices and 0.59 at 200, against 0.53 for the local store. Every decision is covered by the audit batch: in the 200-device runs about 5,000 decisions per run were anchored in roughly 25 transactions. On-chain evaluation costs 44,685 gas as an estimate, but because it is a read, no fee or block wait is incurred.

C. Blockchain Throughput

Table 6 summarises the Besu micro-benchmark. Write confirmation takes 1522 ms on average at 10 tx/s (P95 2439 ms), close to the 2 s block period; a transaction submitted just after a block, as in the phase-locked sequential tests S1/S5, waits about 1980 ms. Reads cost 5.4 ms (P95 10.9 ms). Committed throughput tracks the offered load up to 50 tx/s (46.5 tx/s) but saturates near 60 tx/s on this host: at 100 and 200 tx/s offered only 62.7 and 59.1 tx/s commit and 43% of transactions at 200 tx/s were not confirmed within 60 s. Batch anchoring at 1 tx/s confirmed all 60 anchors in 941 ms on average. Because all validators share one 2-vCPU host, 60 tx/s is a lower bound for a dedicated multi-server deployment.

TABLE 6. MEASURED BESU RESULTS (3 RUNS, MEAN)

Offered (tx/s)	Commit (tx/s)	Mean (ms)	P95 (ms)	Conf.
10	9.5	1522	2439	100%
25	23.3	1706	2659	100%
50	46.5	2330	3626	100%
100	62.7	11601	19753	100%
200	59.1	16424	27044	57%

Per-event logging (S3), four IBFT 2.0 validators on one shared 2-vCPU host; Conf. = confirmed fraction.

D. Resource Utilization

Measured gas per operation is listed in Table 7. Registering a device costs 55,682 gas, whereas committing an epoch root costs 75,469 gas independent of batch size. Logging one event on-chain costs 182,008 gas, while anchoring a batch costs 134,747 gas, i.e. about 135 gas per record for a 1,000-record batch (≈1351× cheaper). At 400 devices and 1 event/s the calculation gives 34.6 M events/day: 6636 MB/day of ledger growth with per-event logging (192 B/entry) versus 13.8 MB/day with batch anchoring. Device-side CPU and memory were not measured; prior IOTA-based DID work reports that DID generation can saturate a Raspberry Pi 4 CPU [22], which motivates our use of hash proofs only.

TABLE 7. MEASURED GAS PER CONTRACT OPERATION

Operation	Gas
IdentityRegistry.registerDID	55,682
IdentityRegistry.commitEpoch	75,469
IdentityRegistry.revokeDID	49,550
AccessControl.setPolicy	49,731
AccessControl.grantSession	71,689
TrustReputation.updateScore (quorum)	124,772
TrustReputation.flagAnomaly	95,868
AuditLog.logEvent (per event)	182,008
AuditLog.anchorBatch (per batch)	134,747

Hardhat EVM (london), optimizer 200 runs; first-use storage costs included.

E. Scalability

Fig. 4 shows goodput and on-chain write load against device count. Central PKI-IAM, ABAC and BC-ZTA served every request up to 200 devices with no failures, whereas the synchronous blockchain saturated: its goodput stopped at 47 req/s (37 req/s at 50 devices), only 74% of requests at 50 devices and 21% at 200 devices were answered in time, mean latency rose to 14.0 s and 2158 requests timed out at 200 devices. Its on-chain write rate (47 tx/s) is the measured ledger ceiling, whereas BC-ZTA writes about 1.0 tx/s regardless of the number of devices. At 200 devices all systems slow down because the PEP, the load generator and the four validators compete for two vCPUs: BC-ZTA reaches 55 ms mean (P95 209 ms) against 12 ms for the local store, so CPU rather than the ledger is its limit on this host. For comparison, a sharded design reaches 2,500 TPS at 700 nodes [18]; our approach instead lowers the required write rate.

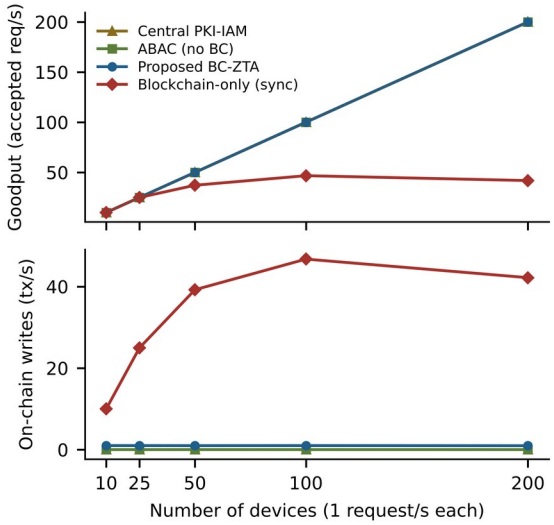


Fig. 4. Goodput (top) and on-chain writes per second (bottom) versus number of devices; dotted line: offered load.

F. Attack Resistance

Table 8 lists the security tests run against the BC-ZTA prototype (3 runs, 50 attempts per test and run). All legitimate requests were accepted (no false rejects), and none of the forged, replayed, stale, unregistered, tampered, class-violating or low-trust requests was accepted. For a revoked device, 0 requests were accepted after the revocation transaction was confirmed, and the first denial came 8 ms after confirmation or sooner. Fig. 5 shows the revocation enforcement delay of each system: with event-driven invalidation BC-ZTA enforces a revocation after 1.22 s on average (worst case 2.12 s), about one block period and independent of the TTL, whereas TTL-only refresh with a 10 s TTL needs 5.5 s on average (worst case 11.2 s). The centralised and local baselines revoke in 48 ms and 5 ms because they hold the state in one place; the blockchain-only value (2.03 s) includes the request that was in flight when the revocation was issued.

TABLE 8. SECURITY TESTS ON THE BC-ZTA PROTOTYPE (3 RUNS)

Test	Attempts	Accepted
Legitimate requests (control)	150	150
Forged signature	150	0
Replayed request	150	0
Stale timestamp	150	0
Unregistered DID	150	0
Tampered Merkle proof	150	0
Class outside policy	150	0
Trust score below threshold	150	0

For the control row, Accepted is the desired outcome; for all other rows it must be 0.

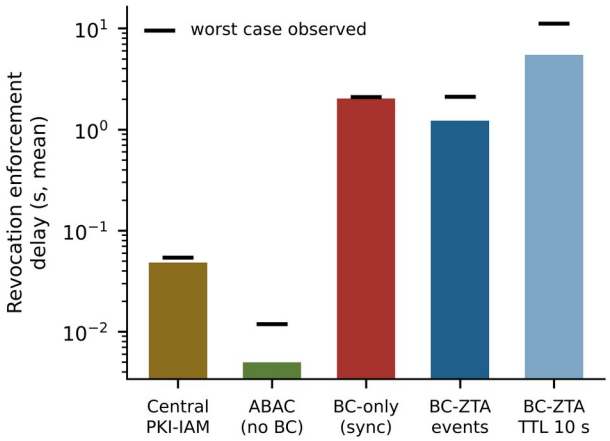


Fig. 5. Revocation enforcement delay (mean, log scale; marks: worst case observed).

G. Comparison with Baselines

BC-ZTA is 3.0 ms slower than a ledger-free ABAC store at 50 devices but adds decentralised identity, shared policy and tamper-evident anchored audit that B2 lacks; against the centralised PKI-IAM it is 85% faster at 50 devices (with an emulated 40 ms WAN) and removes the central directory; and against the synchronous blockchain it avoids ~10.3 s per decision at 50 devices and the ledger ceiling. Its price is revocation latency: about 1.2 s with events (one block) instead of milliseconds for a central store. A reported Ethereum-based framework pays about 25 ms extra per

request even at small scale [17], consistent with our finding that on-path consensus is the bottleneck.

H. Discussion and Limitations

The key trade-off is bounded revocation staleness for latency. Limitations: (i) all four validators, the PEP and the load generator ran on one shared 2-vCPU CI host, so absolute latencies and throughput are not representative of a multi-server plant and validator network delay is not captured; (ii) the WAN delay of the PKI baseline is emulated, and the baselines are simple re-implementations, not production IAM products; (iii) the device population is emulated: no physical devices, industrial protocols or secure elements were used; (iv) one plant authority key issues registrations and revocations, so authority compromise is out of scope; (v) in the flooding test (E4), when 200 tx/s were sent to Besu while 50 devices were served, BC-ZTA success fell to 55–94% (mean latency 1192–8323 ms, against 4.3 ms without flooding), and every failure was a request rejected as stale; we attribute this to CPU starvation of the PEP by the flooded validators on the shared host, so the test does not demonstrate availability under ledger congestion, which needs separate hosts; and (vi) trust scores are voted values, not yet learned from data. Comparisons with published numbers are indicative only, since workloads and testbeds differ.

Deployment considerations. First, validators must be run by independent parties (plant, integrator, vendor) for the Byzantine assumption to mean anything. Second, device keys should live in a secure element or TPM, because the architecture's identity guarantees are only as strong as key custody. Third, on-chain records contain key hashes, scores and event hashes, not process data, which keeps operational data off the shared ledger. Fourth, in the prototype AuditLog and grantSession are callable by any account, and production deployments would restrict them to gateway identities. Fifth, the fail-closed rule of Section III-J trades availability for safety during ledger partitions and should be tuned against plant uptime requirements.

VII. CONCLUSION

We presented BC-ZTA, a blockchain-assisted zero-trust architecture that keeps consensus off the per-request path through edge-cached, Merkle-verified state and asynchronous batched anchoring. Four Solidity contracts pass 15 unit tests, and measured gas shows batch anchoring is about 1351× cheaper per record than per-event logging. In end-to-end tests on a four-validator Besu network, BC-ZTA decided in 7.4 ms on average at 50 devices and served 200 devices without failures with about one on-chain write per second, while the synchronous design needed 10.3 s and saturated at about 47 requests per second; the price is a revocation delay of about 1.2 s with event-driven invalidation. All forged, replayed, stale, unregistered, tampered and policy-violating requests were rejected. The evaluation used one shared host and an emulated device population and WAN link. Future work includes a multi-host Besu deployment with real network delay, physical devices and industrial protocols, learning-based trust scoring and post-quantum signatures.

ACKNOWLEDGMENT

[Funding sources and acknowledgments to be added by the authors.]

REFERENCES

[1] F. Federici, D. Martintoni, and V. Senni, "A zero-trust architecture for remote access in industrial IoT infrastructures," Electronics, vol. 12, no. 3, Art. no. 566, 2023, doi: 10.3390/electronics12030566.

[2] C. Zanasi, S. Russo, and M. Colajanni, "Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures," Ad Hoc Netw., vol. 156, Art. no. 103414, 2024, doi: 10.1016/j.adhoc.2024.103414.

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Spec. Publ. 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[4] M. L. Gambo and A. Almulhem, "Zero trust architecture: A systematic literature review," J. Netw. Syst. Manage., vol. 34, Art. no. 25, 2026, doi: 10.1007/s10922-025-09998-x.

[5] S. Mushtaq, M. Mohsin, and M. M. Mushtaq, "A systematic literature review on the implementation and challenges of zero trust architecture across domains," Sensors, vol. 25, no. 19, Art. no. 6118, 2025, doi: 10.3390/s25196118.

[6] M. A. Azad, S. Abdullah, J. Arshad, H. Lallie, and Y. H. Ahmed, "Verify and trust: A multidimensional survey of zero-trust security in the age of IoT," Internet Things, vol. 27, Art. no. 101227, 2024, doi: 10.1016/j.iot.2024.101227.

[7] S. Mthethwa, M. T. Dlamini, and E. Jembere, "Zero trust for NHIs based on robust identity and access management for a resilient IoT future," Sensors, vol. 26, no. 8, Art. no. 2392, 2026, doi: 10.3390/s26082392.

[8] M. Asim, N. Tariq, A. I. Awad, F. Waheed, S. Bukhari, A. Rafay, and H. H. Song, "Lightweight and privacy-aware decentralized identity management for Industrial Internet of Things applications," Future Gener. Comput. Syst., vol. 186, Art. no. 108753, 2027 (online Aug. 2026), doi: 10.1016/j.future.2026.108753.

[9] E. B. Fernández and A. Brazhuk, "A critical analysis of zero trust architecture (ZTA)," Comput. Stand. Interfaces, vol. 89, Art. no. 103832, 2024, doi: 10.1016/j.csi.2024.103832.

[10] Y.-W. Ma and P.-H. Chiu, "A novel risk-based access control engine in zero trust architecture for IoT network," Int. J. Inf. Secur., vol. 24, Art. no. 124, 2025, doi: 10.1007/s10207-025-01030-2.

[11] M. A. Aleisa, "Blockchain-enabled zero trust architecture for privacy-preserving cybersecurity in IoT environments," IEEE Access, vol. 13, pp. 18660–18676, 2025, doi: 10.1109/ACCESS.2025.3529309.

[12] S. Nie, J. Ren, R. Wu, P. Han, Z. Han, and W. Wan, "Zero-trust access control mechanism based on blockchain and inner-product encryption in the Internet of Things in a 6G environment," Sensors, vol. 25, no. 2, Art. no. 550, 2025, doi: 10.3390/s25020550.

[13] G. Ali, S. Shah, M. Elaffendi, and N. Ahmad, "Blockchain-based access management framework for interoperable digital twins in industrial IoT," Front. Blockchain, vol. 8, Art. no. 1693926, 2025, doi: 10.3389/fbloc.2025.1693926.

[14] M. Essaid and H. Ju, "Blockchain solutions for enhancing security and privacy in Industrial IoT," Appl. Sci., vol. 15, no. 12, Art. no. 6835, 2025, doi: 10.3390/app15126835.

[15] D. Li, H. Li, N. Crespi, R. Minerva, M. Li, W. Liang, and K. C. Li, "Hyper-IIoT: A smart contract-inspired access control scheme for resource-constrained Industrial Internet of Things," IEEE Trans. Sustain. Comput., vol. 10, no. 5, pp. 820–829, 2025, doi: 10.1109/TSUSC.2025.3542466.

[16] Z. Guo, "Blockchain-enhanced smart contracts for formal verification of IoT access control mechanisms," Alexandria Eng. J., vol. 118, pp. 315–324, 2025, doi: 10.1016/j.aej.2024.12.109.

[17] A. Singh, V. Pareek, and A. Sharma, "Blockchain-enabled zero trust framework for securing FinTech ecosystems against insider threats and cyber attacks," arXiv:2507.19976, 2025.

[18] M. M. Salim, M. Kim, S. K. Singh, and J. H. Park, "Zero-trust blockchain-enabled framework for scalable and secure IoT networks," Future Gener.

Comput. Syst., vol. 175, Art. no. 108093, 2026, doi: 10.1016/j.future.2025.108093.

[19] A. Sharma, S. Rani, and W. Boulila, "Blockchain-based zero trust networks with federated transfer learning for IoT security in industry 5.0," PLOS ONE, vol. 20, no. 6, Art. no. e0323241, 2025, doi: 10.1371/journal.pone.0323241.

[20] S. Mishra, T. S. M. Aldafas, and N. S. Alshammari, "A zero-trust digital twin framework for privacy-preserving multi-dataset intrusion detection in industrial IoT with lightweight blockchain auditing," Sci. Rep., vol. 16, Art. no. 15236, 2026, doi: 10.1038/s41598-026-42041-w.

[21] Z. Zhai, J. Liu, X. Liu, Y. Mao, X. Zhang, J. Ma, and C. Jin, "A lightweight authentication method for Industrial Internet of Things based on blockchain and Chebyshev chaotic maps," Future Internet, vol. 17, no. 8, Art. no. 338, 2025, doi: 10.3390/fi17080338.

[22] T. Ramírez-Gordillo, A. Maciá-Lillo, F. A. Pujol, N. García-D'Urso, J. Azorín-López, and H. Mora, "Decentralized identity management for Internet of Things (IoT) devices using IOTA blockchain technology," Future Internet, vol. 17, no. 1, Art. no. 49, 2025, doi: 10.3390/fi17010049.

[23] S. I. R. Naqvi, Z. J. Bin, A. Mohsin, M. P. Akhter, and M. M. ur Rahman, "DBoT: Enhancing scalability and privacy in intelligent building spaces through a direct acyclic graph blockchain," in Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST), 2023, doi: 10.1109/ICOSST60641.2023.10414234.

[24] R. Islam, R. Bose, S. Roy, A. A. Khan, S. Sutradhar, S. Das, F. Ali, and A. A. AlZubi, "Decentralized trust framework for smart cities: A blockchain-enabled cybersecurity and data integrity model," Sci. Rep., vol. 15, Art. no. 23454, 2025, doi: 10.1038/s41598-025-06405-y.

[25] P. Rani, "A zero-trust framework for securing next-generation Internet of Things network," in New Trends and Applications in Internet of Things (IoT) and Big Data Analytics, vol. 2. Cham, Switzerland: Springer, 2026, pp. 213–225, doi: 10.1007/978-3-032-21753-0_12.

[26] B. Chellaprabha, D. Manohari, and J. J. Justus, "Scalable and secure IoT architecture using trust-driven adaptive resource management and edge intelligence," in Expert Applications and Security (NICE-TEAS 2025), Lecture Notes in Networks and Systems, vol. 1823. Cham, Switzerland: Springer, 2026, pp. 444–461, doi: 10.1007/978-3-032-20114-0_40.

[27] K. Li et al., "Zero-trust foundation models: A new paradigm for secure and collaborative artificial intelligence for Internet of Things," arXiv:2505.23792, 2025.

[28] S. Ghasemshirazi, G. Shirvani, and M. A. Alipour, "Zero trust: Applications, challenges, and opportunities," arXiv:2309.03582, 2023.

[29] A. K. Peepliwal et al., "A prototype model of zero-trust architecture blockchain with EigenTrust-based practical Byzantine fault tolerance protocol to manage decentralized clinical trials," arXiv:2408.16885, 2024.

[30] A. Kulkarni, N. A. Hazari, and M. Y. Niamat, "A zero trust-based framework employing blockchain technology and ring oscillator physical unclonable functions for security of field programmable gate array supply chain," IEEE Access, vol. 12, pp. 89322–89338, 2024. [Online]. Available: https://scholars.utoledo.edu/display/pub-001262650600001

AUTHOR BIOGRAPHIES

[AUTHOR NAME] [Short biography, degrees, current position and research interests to be added; IEEE Access also requires a photo.]